

⚙️ Dependency Injection Framework

Vytvořte Dependency Injection (DI) framework, který umožňuje efektivní správu závislostí mezi třídami v aplikaci. Framework poskytuje robustní a flexibilní řešení pro konstrukci komplexních objektových modelů, přičemž využívá moderní designové vzory a osvědčené postupy. Cílem je usnadnit vývojářům práci s injekcí závislostí a zvýšit modularitu a udržitelnost kódu.

Součástí práce je i jednoduchá aplikace, která bude vytvořený framework využívat. Nastavení probíhá prostřednictvím parametrů anotací, případně pomocí konfiguračních souborů.



POZOR: Kratší na vývoj, ale náročnější na návrh!

⚙️ Funkční požadavky

Povinné

1. Práce DIF je řízena pomocí anotací uvnitř tříd.
2. Zdrojové beany jsou ve třídách anotovány pomocí `@Configuration`.
3. Injekci instancí tříd do jiné třídy lze provádět přes konstruktor, setter nebo property. To, kde se injekce použije, určete například anotací `@Autowired`.
4. Lze definovat seznam balíčků (packages), jejichž třídy jsou spravovány DIF (DIF je skenuje).
5. Pokud ve správě DIF existuje více tříd splňujících kontrakt pro injection, implementujte mechanismus pro přesnější určení konkrétní třídy, která se použije.
6. U instancí spravovaných DIF lze nastavit dva typy scope:
 - Singleton – DIF používá stále stejnou instanci.
 - Prototype – DIF při každém požadavku na instanci vrátí novou instanci třídy.
7. DIF lze spustit s různými konfiguracemi – konfigurační soubor například ve formátu `YAML`. V konfiguraci lze specifikovat potřebné parametry (například parametry cache nebo cesty k balíčků, které DIF skenuje).

Bonusové

1. Implementujte tzv. Response Cache. Response cache se nastavuje pomocí anotace `@Cached`, která se přidá před metodu. Takto anotovaná metoda při druhém a dalším volání vrátí hodnotu z cache a tělo metody se neprovolává. Pokud je dosaženo TTL, metoda se provolá znovu.
2. Implementujte `CachingProvider`, tedy rozhraní pro speciální třídu, kterou DIF používá pro implementaci cache. Tento provider automaticky načítá konfiguraci z konfiguračního souboru, kde se nastavuje například

TTL pro cache. Provider poskytuje hotovou instanci komponenty se standardizovaným rozhraním (například pro caching, persistence, logging atd.) a různí dodavatelé dodávají jeho implementaci.

3. Implementujte `CloningProvider`, tedy rozhraní pro speciální třídu, kterou DIF používá pro implementaci klonování tříd. Tento provider automaticky načítá konfiguraci z konfiguračního souboru. Provider bude využit při využití speciálního scope Clone (viz povinné požadavky), který při každém požadavku na instanci vrátí kopii poslední vytvořené instance. Podle konfigurace CloneProvideru se provádí buď shallow copy nebo deep copy.

Nefunkční požadavky

1. Autentizace ani autorizace není požadována.
2. Aplikace může běžet pouze v jedné JVM.
3. Metody a proměnné, které nemají být dostupné ostatním třídám, mají být dobře skryté. Vygenerovaný Javadoc by měl obsahovat co nejméně public metod a proměnných.
4. Reporty jsou generovány do textového souboru.
5. Konfigurace může být načítána přímo z třídy nebo externího souboru (preferován je `JSON`).

Vhodné design patterny

- Dependency Injection 😞
- Factory / Factory method
- Singleton
- Observer
- Chain of Responsibility
- Lazy Initialization
- Object Pool

Doporučená literatura

- <https://martinfowler.com/articles/injection.html>[1]
- <https://docs.spring.io/spring-framework/reference/core/beans.html>[2]
- <https://reflectoring.io/java-annotation-processing/>[3]
- <https://codegym.cc/groups/posts/208-dynamic-proxies>[4]
- <https://python-dependency-injector.ets-labs.org/introduction/index.html>[5]

Hypertext Links

- [1] <https://martinfowler.com/articles/injection.html>
- [2] <https://docs.spring.io/spring-framework/reference/core/beans.html>
- [3] <https://reflectoring.io/java-annotation-processing/>
- [4] <https://codegym.cc/groups/posts/208-dynamic-proxies>
- [5] <https://python-dependency-injector.ets-labs.org/introduction/index.html>

Copyright © 2025 CTU in Prague | Operated by [IT Center of Faculty of Electrical Engineering](#) | Bug reports
and suggestions [ServiceDesk CTU](#)